

Unit 7

Loop Optimization and Pipelining

ECE-GY 6463: ADVANCED HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

Learning Objectives

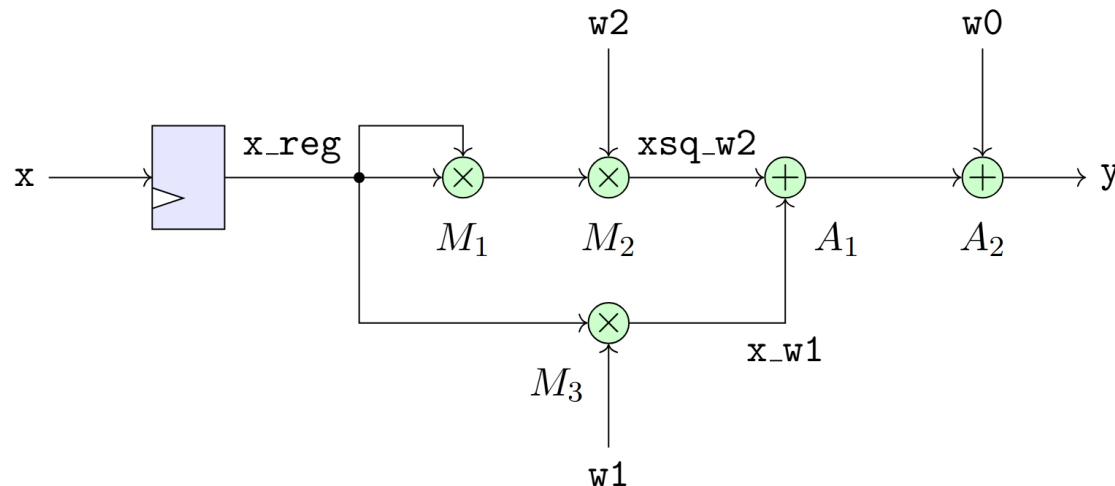
- ❑ Manually **pipeline** simple sequential modules in System Verilog
- ❑ Measure **initiation interval**, **latency**, and total latency
- ❑ Control auto-pipelining in Vitis HLS
- ❑ Extract resource and pipelining data from Vitis HLS **synthesis reports**
- ❑ **Unroll** loops in Vitis HLS
- ❑ Mitigate memory bottlenecks in unrolled loops with:
 - Increased memory bit width
 - **Memory partitioning**

Outline

- ➡ Pipelining in SystemVerilog
 - Loop Pipelining in Vitis HLS
 - Loop Unrolling and Memory Constraints

Quadratic Example

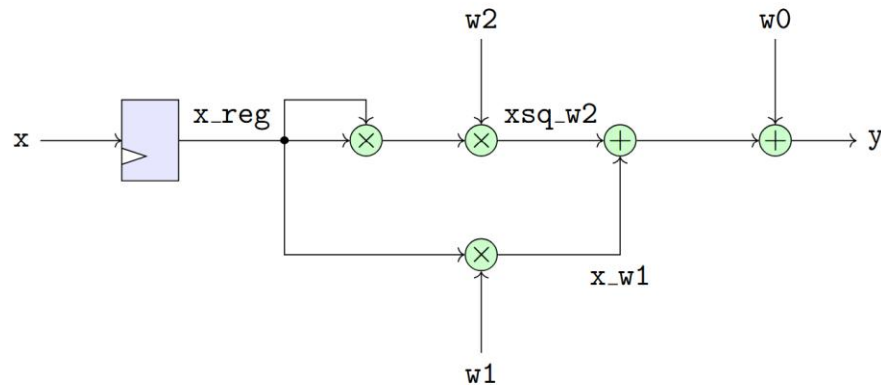
- Consider a quadratic function: $y = w_2x^2 + w_1x + w_0$ (from previous unit)
- For simplicity, assume w_0, w_1, w_2 are fixed parameters
- Single cycle implementation to the right
- Marked multiplier units M_1, M_2, M_3



```
always_ff @(posedge clk) begin
    x_reg <= x;
end
always_comb begin
    xsq = x_reg * x_reg;
    xsq_w2 = w2 * xsq;
    x_w1 = w1 * x_reg;
    y = w0 + xsq_w2 + x_w1;
end
```

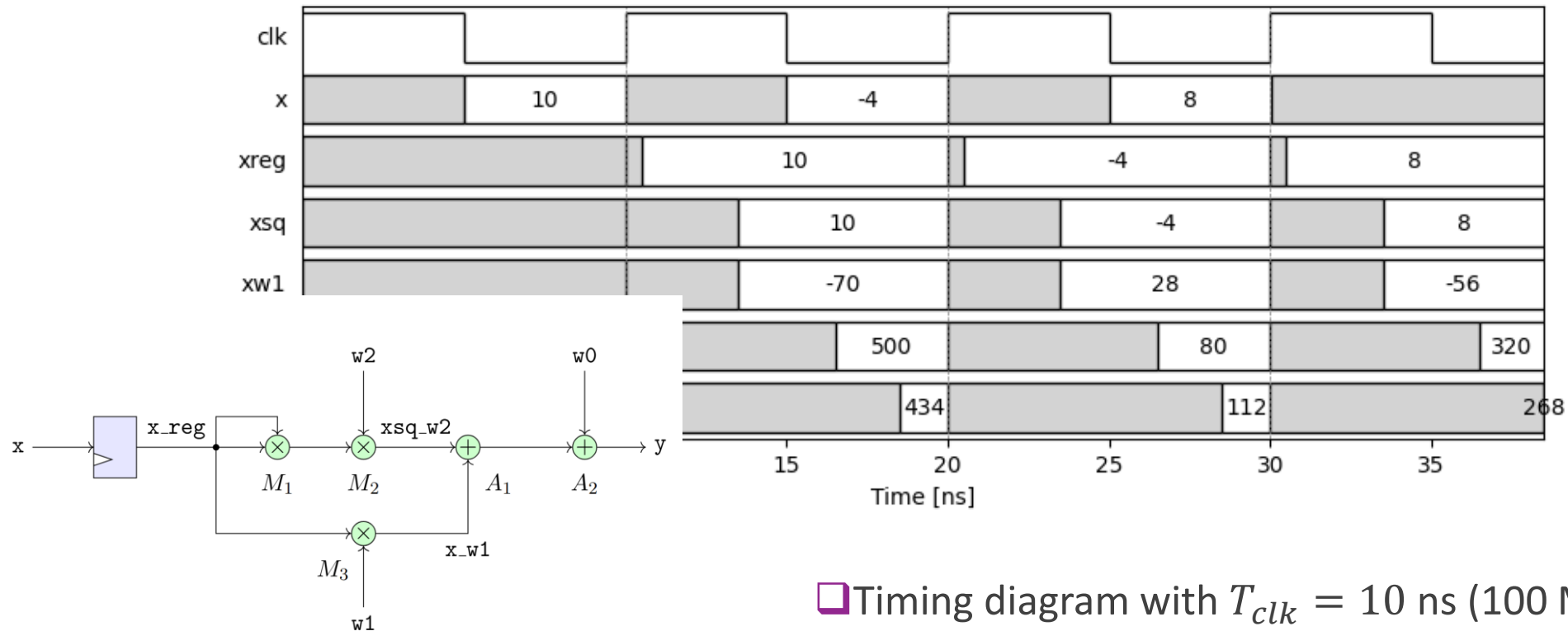
Finding the Critical Path

Hardware block	Value
Multiplier	$T_{mult} = 3 \text{ ns}$
Adder	$T_{add} = 1 \text{ ns}$
Path 1	$3+3+1+1 = 8$
Path 2	$3+1+1=5$



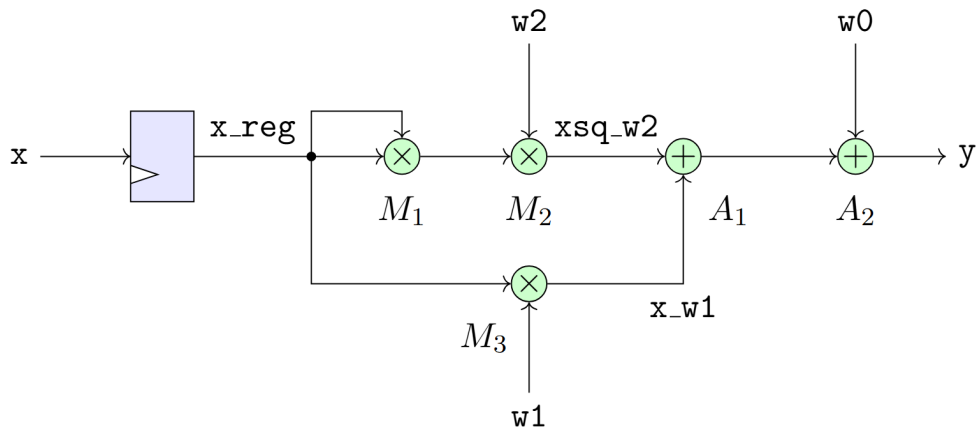
- ❑ Consider implementation with times in table
- ❑ Find critical path
 - Not including clk-Q and setup time
- ❑ Two paths:
- ❑ Path 1: x, x_reg, xsq_w2, y
 - 2 mults, 2 adders = 8.5 ns
- ❑ Path 2: x, x_reg, x_w1, y
 - 1 register, 1 mult, 2 adders = 5.5 ns
- ❑ Path 1 is critical

Timing Diagram



□ Timing diagram with $T_{clk} = 10$ ns (100 MHz)

Multiplier Utilization



❑ When are the multipliers **busy**?

❑ Consider multiplier M_1 :

- Input is x_reg available 0.5 ns after clock edge
- Max PD for multiplier is $T_{mult} = 3$ ns
- So M_1 is busy for [0.5, 3.5] ns in each interval

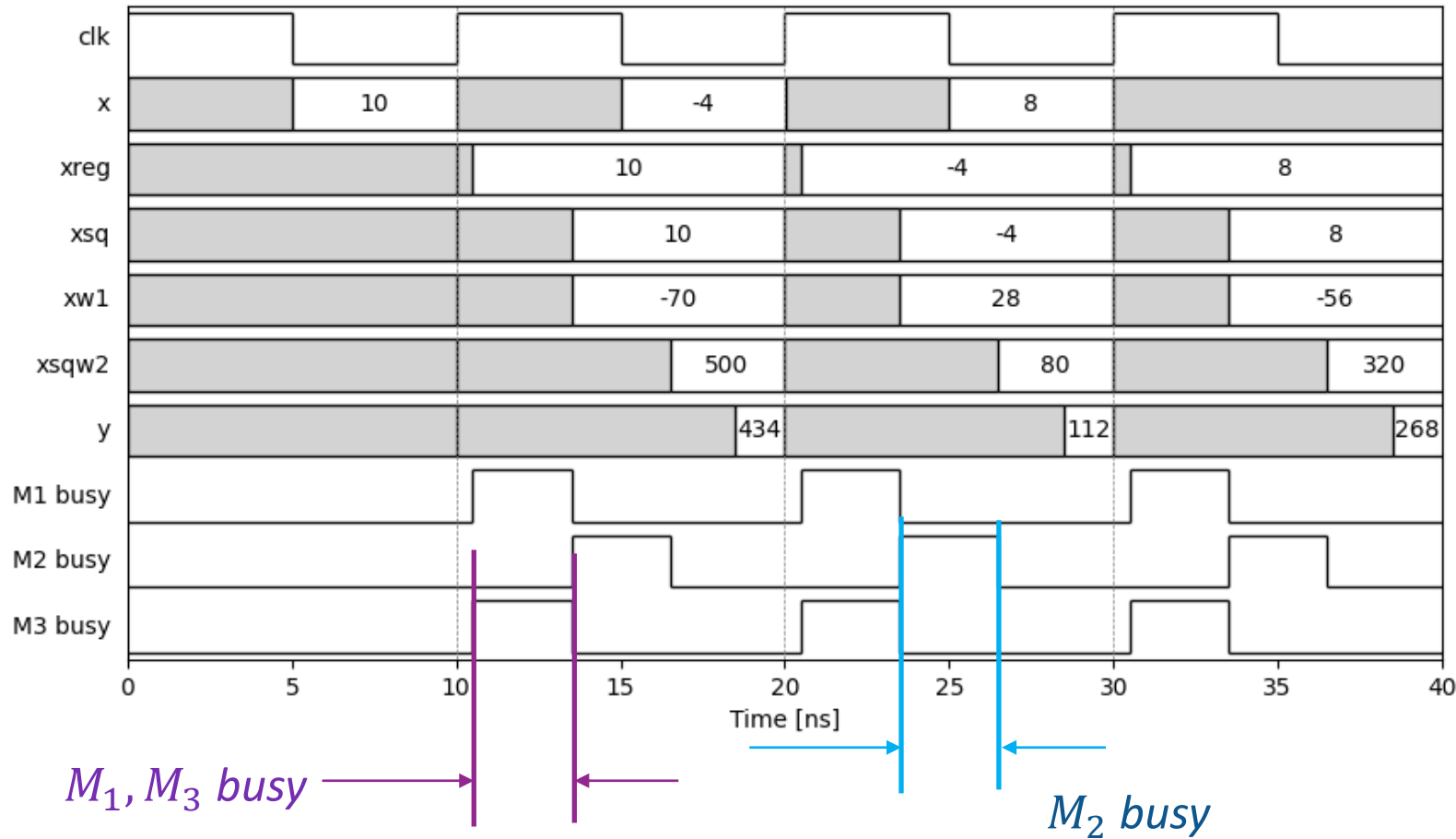
❑ Rest of the time, M_1 is **idle**

- Not being used. “Waiting” for next input

❑ Fraction **utilization**:

$$\text{utilization} = \frac{\text{Busy time}}{T_{clk}} = \frac{3}{10} = 30\%$$

Plotting the Utilization



Each multiplier busy only once in clock period

Best Utilization with Single Cycle Design

❑ From previous slide:

- Critical path delay $T_{crit} = 8 \text{ ns}$
- Two multipliers + two adders

❑ To close timing, assume parameters to right:

- Find min clock period
- $T_{clk} \geq T_{crit} + T_{setup} + T_{clk-Q} = 9.5 \text{ ns}$

❑ Each multiplier is busy T_{mult} per clock period

❑ Best utilization

$$Utilization = \frac{T_{mult}}{T_{clk}} \leq \frac{3}{9.5} \approx 0.31$$

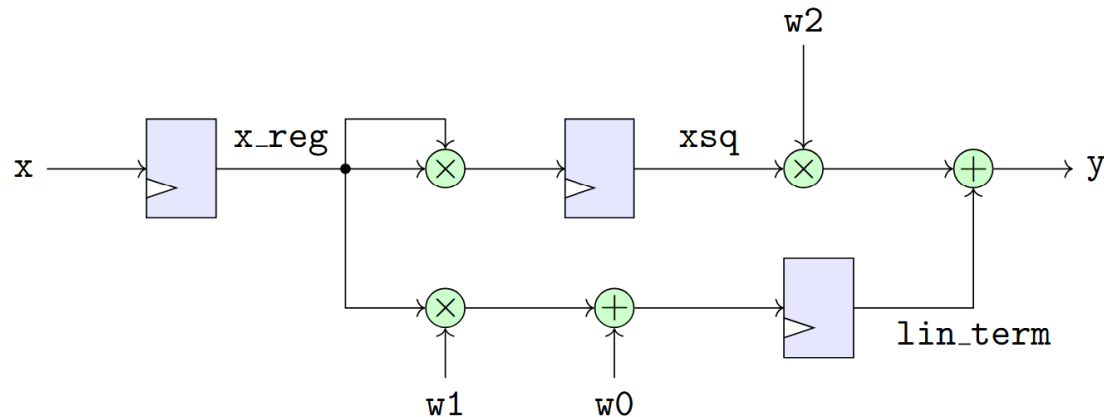
Item	Value
Register (clk-Q) delay	$T_{clk-Q} = 0.5 \text{ ns}$
Multiplier PD	$T_{mult} = 3 \text{ ns}$
Adder PD	$T_{add} = 1 \text{ ns}$
Min set up time	$T_{setup} = 1 \text{ ns}$
Critical path delay (in combinational logic)	$T_{crit} = 3+3+1+1 = 8$
Min T_{clk}	$T_{clk} = 8.5+0.5+1 = 9.5 \text{ ns}$

A Two Cycle Design

❑ We can improve throughput by breaking computation over two cycles

❑ Register two intermediate terms:

- $xsq = xreg * xreg$
- $lin_term = w1 * xreg + w0$



```
int xreg;
always_ff @(posedge clk) begin
    xsq <= xreg * xreg;
    lin_term <= w1 * xreg + w0;
    xreg <= x;
end
always_comb begin
    y = w2*xsq + lin_term;
end
```

Three Stages

□ Design is processed in three stages = 2 cycles

□ Stage 0

- Register input x to `xreg`

□ Stage 1: Compute and register:

- `xsq = xreg * xreg`
- `lin_term = w1*xreg + w0`

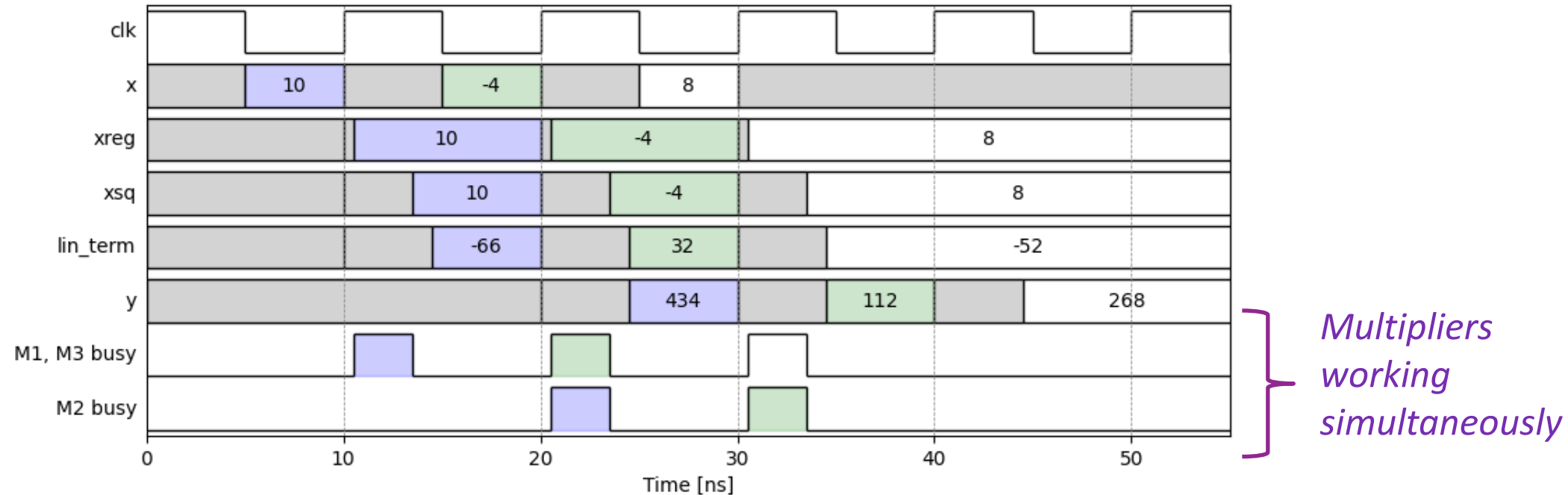
□ Stage 2: Compute output

- `y = w2*xsq + lin_term`

```
int xreg;
always_ff @(posedge clk) begin
    xsq <= xreg * xreg;
    lin_term <= w1 * xreg + w0;
    xreg <= x;
end
always_comb begin
    y = w2*xsq + lin_term;
end
```

Pipelining Timing Diagram

Processing via a “pipeline”



Multipliers
working
simultaneously

Comparison

		Single Cycle Design	Two Cycle Design
Register + Setup time	$T_{clk-Q} + T_{setup}$	1.5	
Multiplier PD	T_{mult}	3	
Add PD	T_{add}	1	
Critical path PD	T_{crit}	3+3+1+1=8	3+1=4
Min clock period	T_{clk}	9.5	5.5
Max throughput	$1/T_{clk}$	105 MHz	181 MHz
Latency		$T_{clk} = 9.5$	$2T_{clk} = 11$
Multiplier utilization	T_{mult}/T_{clk}	0.31	0.54

Pipelining: Higher throughput via better utilization

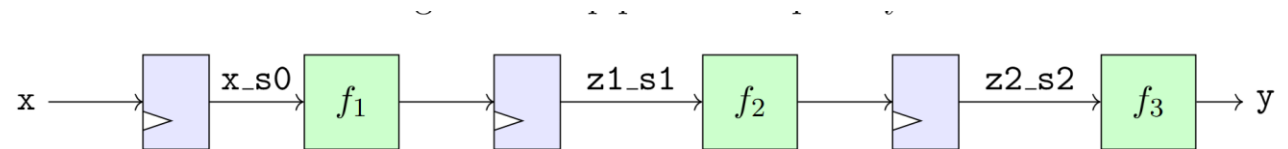
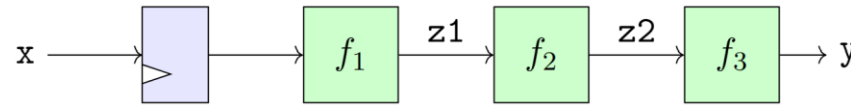
General Pipelining for a Series

```
always_ff @(posedge clk) begin
    x_reg <= x;
end
always_comb begin
    z1 = f1(x_reg);
    z2 = f2(z1);
    y = f3(z2);
end
```

```
always_ff @(posedge clk) begin
    x_s0 <= x;
    z1_s1 <= f1(x_s0);
    z2_s2 <= f2(z1_s1);
end
always_comb begin
    y = f3(z2_s2);
end
```

❑ Insert registers on each stage

❑ Use suffixes s0, s1, s2 to indicate stage

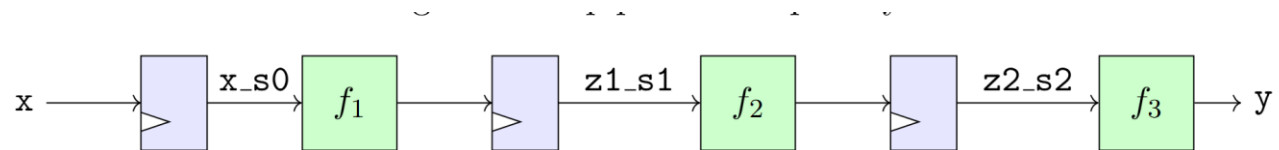


Cycle-by-Cycle Timing

Cycle	x	x_s0	$z1_s1$	$z2_s2$	y
0	A	?	?	?	?
1	B	A	?	?	?
2	C	B	$f_1(A)$	?	?
3	D	C	$f_1(B)$	$f_2(f_1(A))$	$f_3(f_2(f_1(A)))$
		D	$f_1(C)$	$f_2(f_1(B))$	$f_3(f_2(f_1(B)))$
			$f_1(D)$	$f_2(f_1(C))$	$f_3(f_2(f_1(C)))$
				$f_2(f_1(D))$	$f_3(f_2(f_1(D)))$

```

always_ff @(posedge clk) begin
    x_s0 <= x;
    z1_s1 <= f1(x_s0);
    z2_s2 <= f2(z1_s1);
end
always_comb begin
    y = f3(z2_s2);
end
    
```



Timing Terminology

❑ Initiation Interval (II)

- Minimum number cycles between the start of two inputs entering the pipeline

❑ Latency (L)

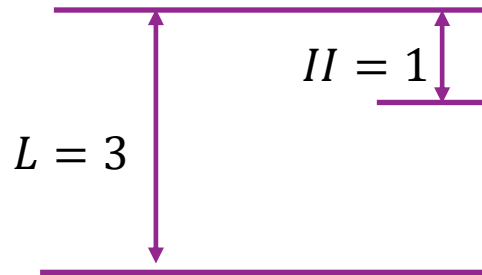
- Number of cycles between an input and when output is available
- If input is available before end of cycle n , output available before end of cycle $n + L$
- Equal to number of stages between registered inputs

❑ Number of samples (n)

- Number of consecutive input samples

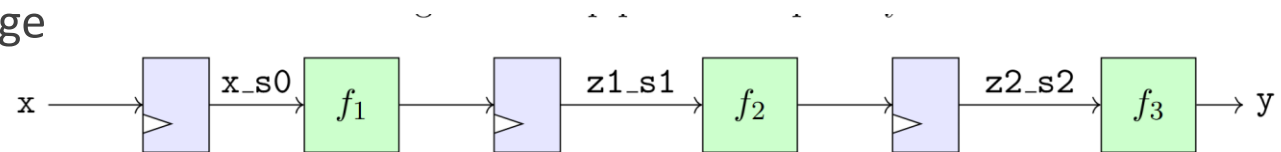
❑ Total processing time: $T = (n - 1) \times II + L$

Timing Terminology Visualized



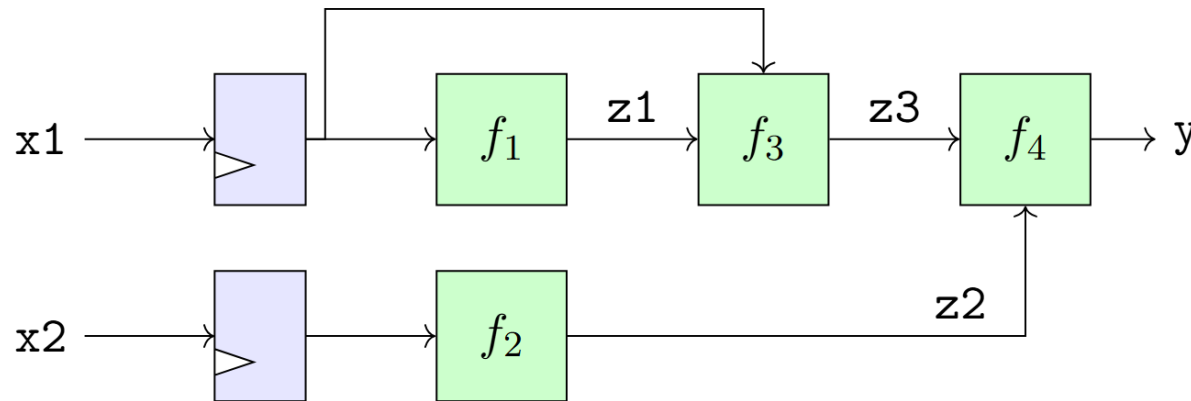
Cycle	x	x_{s0}	$z1_{s1}$	$z2_{s2}$	y
0	A	?	?	?	?
1	B	A	?	?	?
2	C	B	$f_1(A)$	?	?
3	D	C	$f_1(B)$	$f_2(f_1(A))$	$f_3(f_2(f_1(A)))$
		D	$f_1(C)$	$f_2(f_1(B))$	$f_3(f_2(f_1(B)))$
			$f_1(D)$	$f_2(f_1(C))$	$f_3(f_2(f_1(C)))$
				$f_2(f_1(D))$	$f_3(f_2(f_1(D)))$

- ❑ Insert registers on each stage
- ❑ Use suffixes s0, s1, s2 to indicate stage



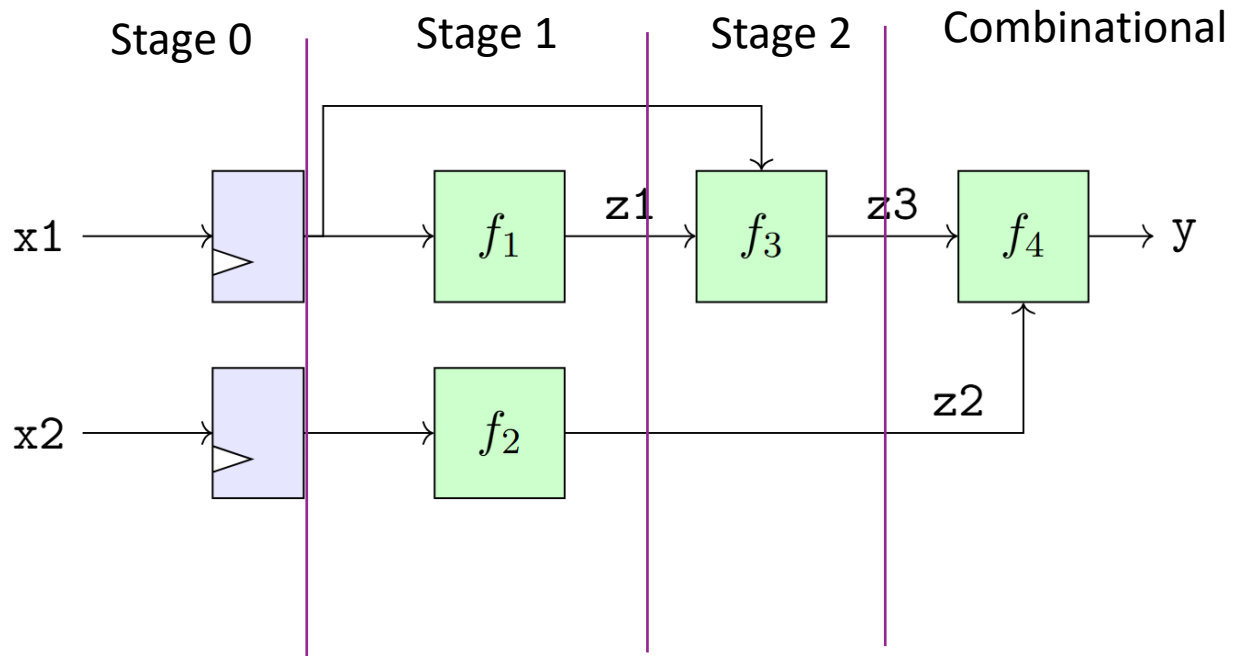
Pipelining with Multiple Paths

- Now consider a more complex example with multiple paths
- How do we pipeline this?



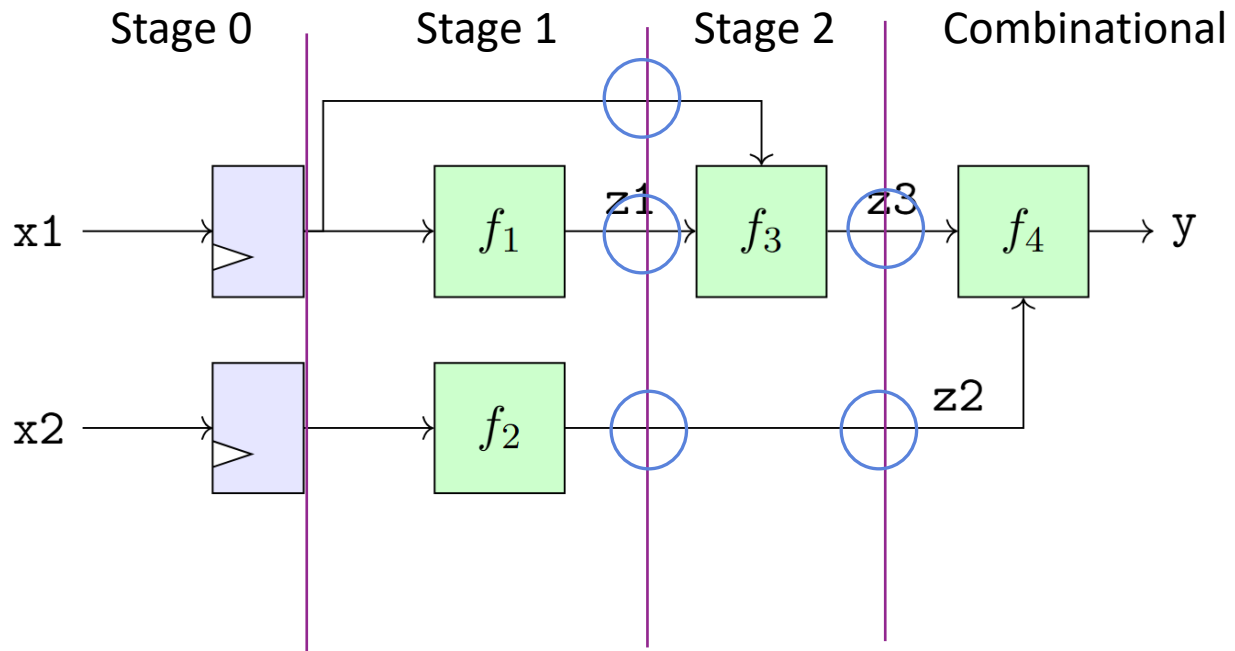
```
always_ff @(posedge clk) begin
    x1_reg <= x1;
    x2_reg <= x2;
end
always_comb begin
    z1 = f1(x1_reg);
    z2 = f2(x2_reg);
    z3 = f3(z1, x1_reg);
    y = f4(z2, z3);
end
```

Multi-Path Pipelining: Identify Stages



- ❑ First, we draw the stages
 - Shown in the vertical lines
- ❑ Stage 0: Inputs to registers
 - This stage will exist in non-pipeline code
- ❑ Stage 1, 2, ...
 - Will end where we will put registers
 - Ensure that each stage can be computed in sufficient time
- ❑ Output stage is combinational

Multi-Path Pipelining: Register Locations



Place registers at stage crossing

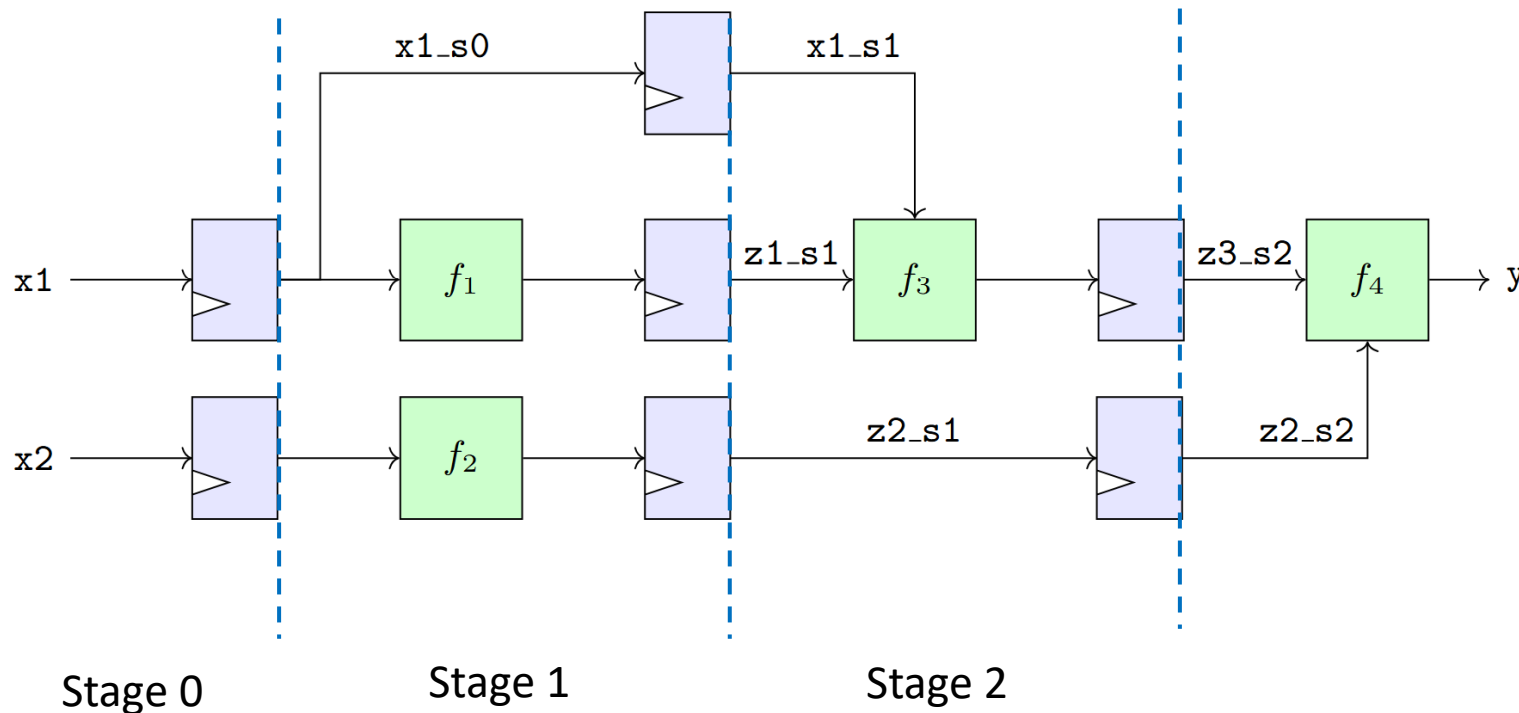
Shown in blue circles

Note: Need register even when

- Signal is just a pass through

Register locations

Final Pipelined Diagram and Code



```
always_ff @(posedge clk) begin
    x1_s0 <= x1;
    x2_s0 <= x2;
    z1_s1 <= f1(x1_s0);
    z2_s1 <= f2(x2_s0);
    x1_s1 <= x1_s0;
    z3_s2 <= f3(z1_s1, x1_s1);
    z2_s2 <= z2_s1;
end
always_comb begin
    y = f4(z2_s2, z3_s2);
end
```

Memory Accesses

- ❑ Memory reads and writes:
 - Generally take at least one cycle delay
 - Hardware provides an address
 - Memory location fetched with an address decoder
- ❑ Pipelining must account for this delay
- ❑ Example to the right:
 - Takes elementwise product
 - Load, product, and store are on separate cycles
- ❑ Note the pipeline flush
 - Tail of the pipeline for stages 1,

```
logic [31:0] x[0:N-1], w[0:N-1];
logic [31:0] y[0:N-1];

logic [8:0] i;
always_ff @(posedge clk) begin
    // Some start-stop logic
    // running = ...

    // Stage 0: Read x[i] and w[i]
    if ((running) && (i < N)) begin
        x_s0 <= x[i];
        w_s0 <= w[i];
    end

    // Stage 1: Compute product
    if ((running) && (i >= 1) && (i < N+1)) begin
        prod_s1 <= x_s0 * w_s0;
    end

    // Stage 2: Write output
    if ((running) && (i >= 2) && (i < N+2)) begin
        y[i-2] <= prod_s1;
    end

    if ((running) && (i < N+2)) begin
        i <= i + 1;
    end
end
```

Outline

☐ Pipelining in SystemVerilog

 ☐ Loop Pipelining in Vitis HLS

☐ Loop Unrolling and Memory Constraints

Why Use Vitis HLS?

```
logic [31:0] x[0:N-1], w[0:N-1];
logic [31:0] y[0:N-1];

logic [8:0] i;
always_ff @(posedge clk) begin
    // Some start-stop logic
    // running = ...

    // Stage 0: Read x[i] and w[i]
    if ((running) && (i < N)) begin
        x_s0 <= x[i];
        w_s0 <= w[i];
    end

    // Stage 1: Compute product
    if ((running) && (i >= 1) && (i < N+1)) begin
        prod_s1 <= x_s0 * w_s0;
    end

    // Stage 2: Write output
    if ((running) && (i >= 2) && (i < N+2)) begin
        y[i-2] <= prod_s1;
    end

    if ((running) && (i < N+2)) begin
        i <= i + 1;
    end
end
```

```
for (int i=0; i < N; i++) {
    #pragma HLS PIPELINE II=1
    y[i] = w[i]*x[i];
}
```

❑ Manual pipelining in System Verilog:

- Tedious
- Prone to mistakes
- Hard to optimize schedule
- Need to re-optimize on each technology

❑ Vitis HLS C++

- Fully automated pipelining
- High-level description

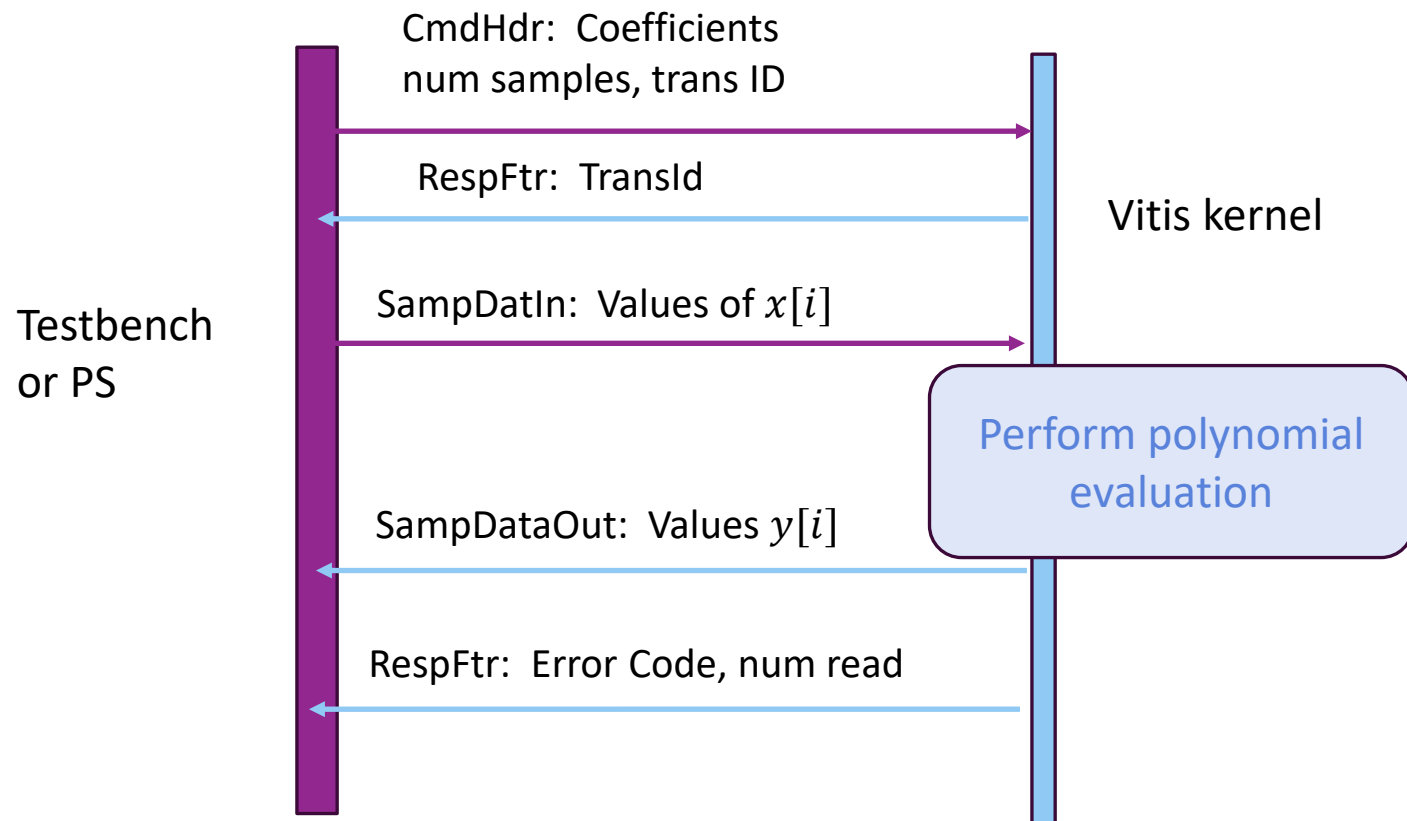
Polynomial Loop Example

- ❑ Loop over cubic polynomial
- ❑ Implemented with Horner's method
 - Bottom right
- ❑ Each iteration involves
 - One load from the array
 - Three multiplications
 - Three adds
 - One store
- ❑ We will implement in floating point FP32
 - These are themselves pipelined
- ❑ Would be hard to implement manually

```
for (i=0; i < n; i++) {  
    y[i] = a[0] + a[1]*x[i]  
          + a[2]*x[i]*x[i] + a[3]*x[i]*x[i]*x[i];  
}
```

```
for (i=0; i < n; i++) {  
    xi = x[i];  
    yi = a[3];  
    yi += xi*yi + a[2];  
    yi += xi*yi + a[1];  
    yi += xi*yi + a[0];  
    y[i] = yi;  
}
```

Message Passing Protocol



- ❑ Similar protocol to previous unit
- ❑ Processing in blocks
- ❑ Coefficients sent in a header
- ❑ Followed by data
- ❑ Transaction ID
 - Used to ensure synchronization
 - Robust to dropped packets

Define Data Structures

- ❑ Use xilinxutils package
- ❑ Define data structures in python

```
class PolyCmdHdr(DataList):
    elements = {
        "tx_id": {
            "schema": TransId,
            "description": "Transaction ID",
        },
        "coeffs": {
            "schema": CoeffArray,
            "description": "Polynomial coefficients",
        },
        "nsamp": {
            "schema": Nsamp,
            "description": "Number of samples",
        },
    }
    include_dir = INCLUDE_DIR
```

```
class CoeffArray(DataArray):
    ncoeffs = 4
    element_type = Float32
    static = True
    max_shape = (ncoeffs,)
    include_dir = INCLUDE_DIR
```

```
TransId = IntField.specialize(bitwidth=16, signed=False)
Nsamp = IntField.specialize(bitwidth=16, signed=False)
```

Create a Python Golden Model

- ❑ Python Golden model
- ❑ Simulates the same messages
- ❑ Runs python polynomial evaluation

```
def polynomial_eval(  
    cmd_hdr: PolyCmdHdr,  
    samp_in: SampDataIn,  
) -> tuple[PolyRespHdr, SampDataOut, PolyRespFtr]:  
    resp_hdr = PolyRespHdr()  
    resp_hdr.tx_id = cmd_hdr.tx_id  
  
    coeffs = np.asarray(cmd_hdr.coeffs, dtype=np.float32)  
    x = np.asarray(samp_in.val, dtype=np.float32)  
    y = np.zeros_like(x)  
    power = np.ones_like(x)  
    for coeff in coeffs:  
        y += coeff * power  
        power *= x  
  
    samp_out = SampDataOut()  
    samp_out.val = y  
  
    resp_ftr = PolyRespFtr()  
    resp_ftr.nsamp_read = len(x)  
    if len(x) == int(cmd_hdr.nsamp):  
        resp_ftr.error = PolyError.NO_ERROR  
    else:  
        resp_ftr.error = PolyError.WRONG_NSAMP  
  
    return resp_hdr, samp_out, resp_ftr
```

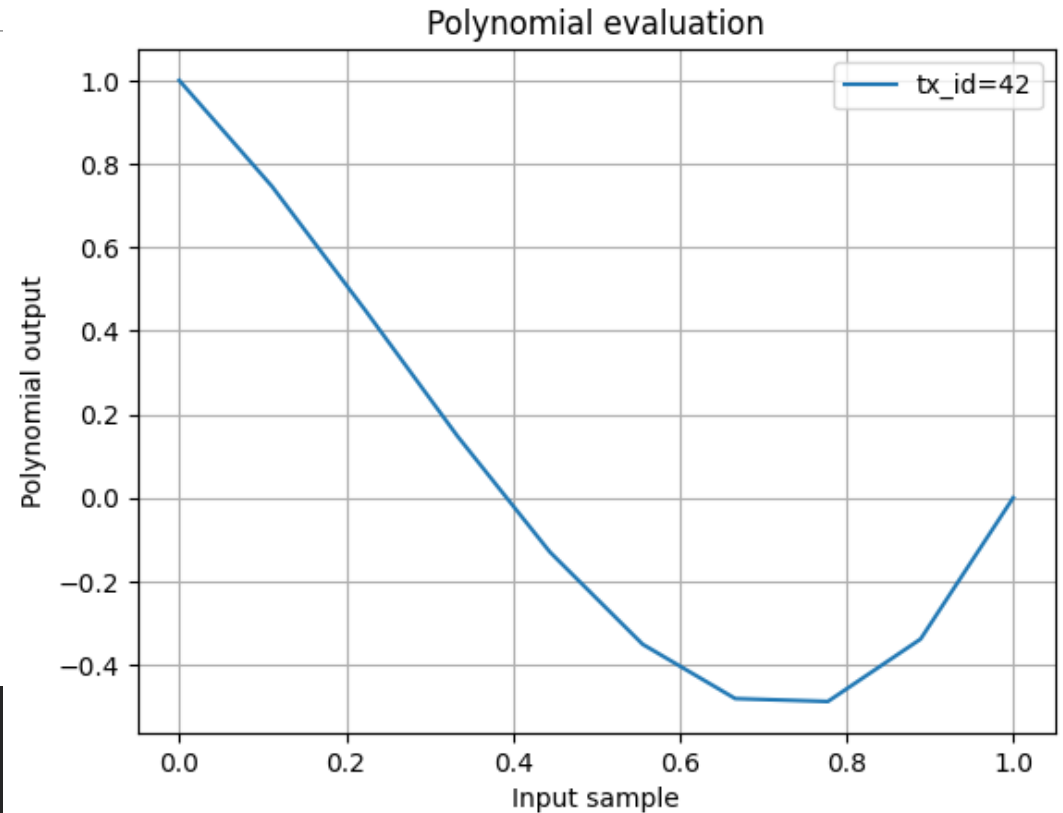
Running the Golden Model

❑ Run the Golden model

❑ Generate:

- Inputs
- Expected outputs

```
data_dir = Path.cwd() / "data"
data_dir.mkdir(exist_ok=True)
cmd_hdr.write_uint32_file(data_dir / "cmd_hdr_data.bin")
samp_in.write_uint32_file(data_dir / "samp_in_data.bin", nwrite=cmd_hdr.nsamp)
samp_out.write_uint32_file(data_dir / "samp_out_data.bin", nwrite=resp_ftr.nsamp_read)
```



Auto-Generate the Header Files

- ❑ For each class, we generate a header file
- ❑ Header files include:
 - Fields in the data structure
 - Methods for serializing and deserializing

```
struct PolyCmdHdr {
    ap_uint<16> tx_id; // Transaction ID
    CoeffArray coeffs; // Polynomial coefficients
    ap_uint<16> nsamp; // Number of samples

    static constexpr int bitwidth = 160;

    template<int word_bw>
    static constexpr int nwords() {
        if constexpr (word_bw == 32) {
            return 6;
        }
        else if constexpr (word_bw == 64) {
            return 3;
        }
        else {
            static_assert(word_bw > 0, "Unsupported word width");
            return 0;
        }
    }
}
```

Vitis Kernel Implementation

```
static float x[MAX_NSAMP], y[MAX_NSAMP];

// Read to the input buffer
load_loop: for (int i=0; i < (int)cmd_hdr.nsamp; ++i) {
    auto word = in_stream.read();
    x[i] = streamutils::uint_to_float(word.data);
}

proc_loop: for (int i = 0; i < cmd_hdr.nsamp; i++ ) {
#pragma HLS PIPELINE II=1
    y[i] = eval_poly_horner(cmd_hdr.coeffs.data, x[i]);
}

// Write the output to the stream
store_loop: for (int i=0; i < (int)cmd_hdr.nsamp; ++i) {
    axis_word_t word;
    word.data = streamutils::float_to_uint(y[i]);
    word.last = (i == cmd_hdr.nsamp - 1) ? 1 : 0;
    out_stream.write(word);
}
```

❑ Use array:

- Load to array, compute, write array to stream
- Computationally, adds an extra step
- But allows easy loop optimization analysis

❑ Main loop is very simple

❑ Note pragma HLS PIPELINE II=1

- Indicates to Vitis to try to pipeline this loop
- Try to find a schedule with II=1

Running Vitis CS

- ❑ Run C-simulation
 - Performs functional validation
- ❑ Run C-synthesis
 - Performs pipelining
 - Mapping of functions to resources
- ❑ File to right: TCL script

```
open_project -reset poly_proj
set_top poly
add_files poly.cpp
add_files -tb poly_tb.cpp

open_solution -reset "solution1"
set_part {xc7z020clg484-1}
create_clock -period 10

set script_dir [file dirname [file normalize [info script]]]
set data_dir [file join $script_dir "data"]

if {[catch {csim_design -argv "$data_dir"} res]} {
    puts "FAIL: poly C-Simulation failed."
    puts $res
    exit 1
}

puts "SUCCESS: poly C-Simulation passed."

if {[catch {csynth_design} res]} {
    puts "FAIL: poly C-Synthesis failed."
    puts $res
    exit 1
}

puts "SUCCESS: poly C-Synthesis passed."
exit 0
```

Parsing the Results

- ❑ Vitis loop optimization results stored in XML file
- ❑ Can parse results in xilinxutils CsynthParser
- ❑ Allows easy automation

```
from xilinxutils.csynthparse import CsynthParser
import os

# Parse the synthesis for UF=1
sol_path = os.path.join(os.getcwd(), 'poly_proj', 'solution1')
parser = CsynthParser(sol_path=sol_path)

# Get the latency and initiation interval
print('Latency and Initiation Interval:')
parser.get_loop_pipeline_info()
print(parser.loop_df)

# Get the resources
print('\nResource Usage:')
parser.get_resources()
print(parser.res_df)

0.0s
```

Pipelining Results

- Main processing loop
 - Obtains $II=1$
 - High latency $L=30$
 - Needed for many FP32 multiplies

Latency and Initiation Interval:

	PipelineII	PipelineDepth
poly_Pipeline_VITIS_LOOP_262_1:VITIS_LOOP_262_1	1	2
poly_Pipeline_load_loop:load_loop	1	2
poly_Pipeline_proc_loop:proc_loop	1	30
poly_Pipeline_store_loop:store_loop	1	2

Resource Usage:

	BRAM_18K	DSP	FF	LUT	URAM
poly_Pipeline_VITIS_LOOP_262_1	0	0	136	73	0
poly_Pipeline_load_loop	0	0	19	87	0
poly_Pipeline_proc_loop	0	15	1471	2273	0
poly_Pipeline_store_loop	0	0	12	93	0
poly	2	15	1770	2830	0
Total	2	15	1770	2830	0
Available	280	220	106400	53200	0

Loop Carry Dependency

❑ Not all loops can be pipelined

❑ Consider merge sort

- Given two sorted lists $x_1[i]$, $x_2[j]$
- Find a merged sorted list $y[k]$

❑ Algorithm:

- C code to the right
- Compare current $x_1[i]$ and $x_2[j]$
- Places smaller one on list
- Increments counter for the selected element

```
for (i=0, j=0, k=0; i < N1 && j < N2; k++) {  
    if (i == N1) {  
        y[k] = x2[j];  
        j++;  
    } else if (j == N2) {  
        y[k] = x1[i];  
        i++;  
    } else if (x1[i] < x2[j]) {  
        y[k] = x1[i];  
        i++;  
    } else {  
        y[k] = x2[j];  
        j++;  
    }  
}
```

Merge Sort Example

k	i	j	$x_1[i]$	$x_2[j]$	$y[k]$
0	0	0	1	2	1
1	1	0	4	2	2
2	1	1	4	3	3
3	1	2	4	6	4
4	2	2	10	6	6
5	2	3	10	X	10

$x_1[i]$	$x_2[j]$
1	2
4	3
10	6

Pipelining Limit

- ❑ Consider main loop body needed to the right
- ❑ Stage 0 of iteration t
 - Depends on stage 1 completion in iteration $t - 1$
 - Why? Since addresses i and j are updated
- ❑ Implication: Each iteration will need:
 - Stage 0 iteration t
 - Stage 1 iteration t
 - Stage 0 iteration $t + 1$
- ❑ Minimum $II=2$
- ❑ Called a **loop carry dependency**

```
// stage 0  
x1i <= x1[i];  
x2j <= x2[j];
```

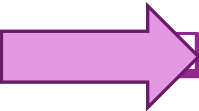
```
// stage 1  
if (x1i < x2j) begin  
    yk <= x1i;  
    i <= i + 1;  
end else begin  
    yk <= x2j;  
    j <= j + 1;  
end
```

```
// stage 2  
y[k] <= yk;  
k <= k+1;
```

Outline

□ Pipelining in SystemVerilog

□ Loop Pipelining in Vitis HLS

 □ Loop Unrolling and Memory Constraints

Loop Unrolling

- ❑ Simple idea: Repeat processing in each cycle
- ❑ Unroll by 2
 - Need half the number of iterations
 - But double the logic

```
// Normal loop  
for (i=0; i < N; i++) {  
    y[i] = w[i]*x[i];  
}
```

```
// Unrolled by factor of 2  
for (i=0; i < N; i+=2) {  
    y[i] = w[i]*x[i];  
    y[i+1] = w[i+1]*x[i+1];  
}
```

Loop Unrolling in Vitis

- ❑ Can be done with a simple pragma
- ❑ No need to manually unroll!
- ❑ Handles cases when N is not divisible by unroll factor

```
// Vitis pragma
for (i=0; i < N; i++) {
    #pragma HLS unroll factor=2
    y[i] = w[i]*x[i];
}
```

=

```
// Unrolled by factor of 2
for (i=0; i < N; i+=2) {
    y[i] = w[i]*x[i];
    y[i+1] = w[i+1]*x[i+1];
}
```

Memory Bottleneck

// Stage 0: Two reads from x and w

```
x1_s0 <= x[i];  
x2_s0 <= x[i+1];  
w1_s0 <= w[i];  
w2_s0 <= w[i+1];
```

// Stage 1: Two products -> OK

```
y1_s1 <= x1_s0 * w1_s0;  
y2_s1 <= x2_s0 * w2_s0;
```

// Stage 2: Two writes

```
y[i] <= y1_s1;  
y[i+1] <= y2_s1;  
i <= i + 2;
```

❑ Memory bottleneck

- Loop unrolling is often limited by memory accesses

❑ Consider vector multiply SV code

❑ Stage 0 needs:

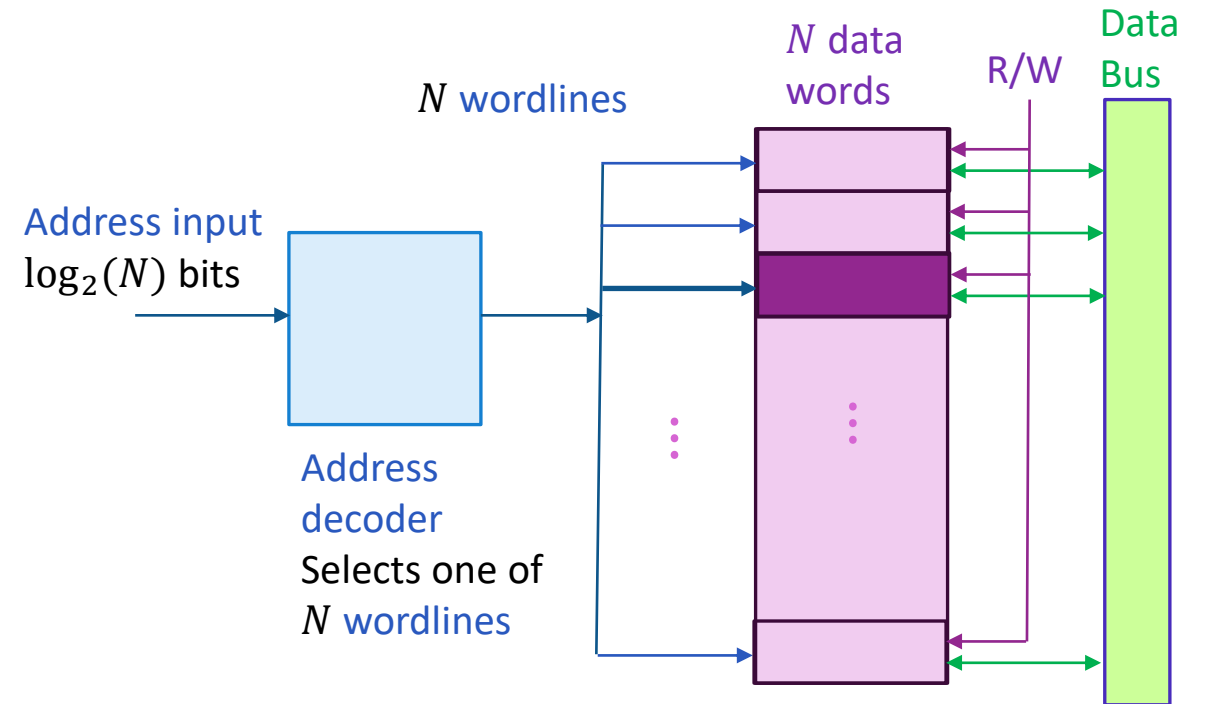
- Two reads from x and two reads from w

❑ Stage 2 needs:

- Two writes to y

Single Port Memory

- Recall memory architecture from Unit 4
- Basic memories are **single port**
- There is a single address decoder
- Can read / write one word in a cycle



Three Solutions

❑ Solution 1: Use a true multi-port RAM

- Most general. Allows read and write from any combination of addresses
- Xilinx FGPA BRAMs are two port
- But most costly in hardware

❑ Solution 2: Use a wider bitwidth

- Say 64 bits to read two 32 bit numbers
- Works only when the accesses are contiguous

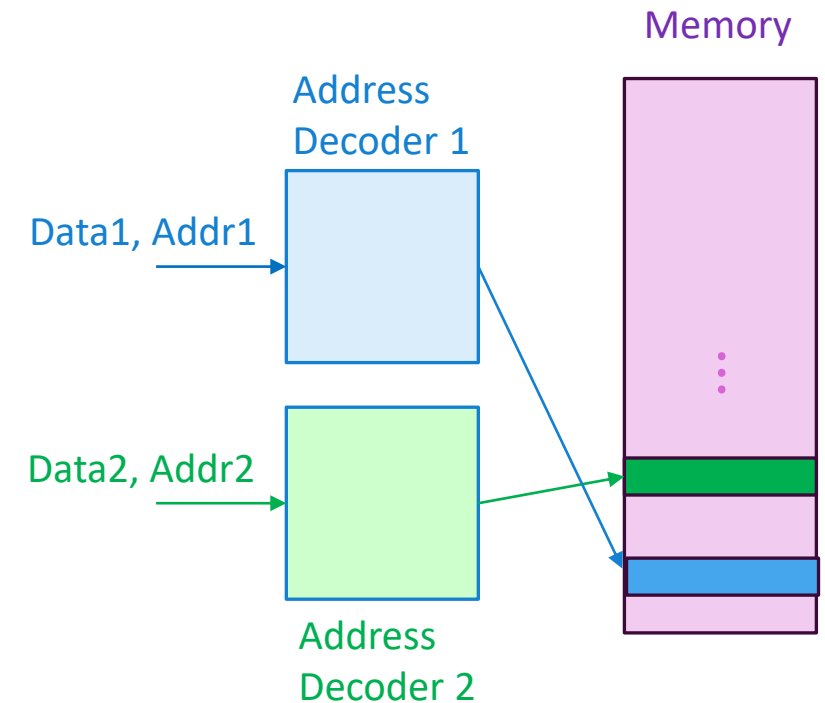
❑ Solution 3: Bank or partition memory

- Divide memory into partitions
- Each partition has its own decoder
- Works if accesses can be across different partitions

Mutli-Port RAMs

- ❑ Allow arbitrary read / writes
 - Except for read-write collisions to same address in same cycle
- ❑ BRAM in FPGAs generally only support true two port
- ❑ BMutli-port SRAMs in ASICs are expensive

Logical ports	Typical cell style	Rough transistors/bit	Rough area vs 6T
1R/1W (single-port)	6T SRAM cell	~6T	1.0×
1R/1W (2-port: 1R1W)	8T cell (separate read path)	~8T	≈1.3×
2R/1W	10T–12T cell	~10–12T	≈1.7–2.0×
2R/2W	12T–16T+ cell	~12–16T+	≈2–2.7×
N-port (general)	custom multi-port SRAM	≈6T + O(N·ports)	grows ~linear in ports



Using Wider Bit widths

Narrow bit width design

- Access only one word per cycle
- Memory bottleneck

```
// Unrolled by factor of 2
for (i=0; i < N; i+=2) {
    y[i] = w[i]*x[i];
    y[i+1] = w[i+1]*x[i+1];
}
```

```
ap_uint<64> xwords[N/2], wwords[N/2], ywords[N/2];

for (i=0; i < N; i += 2) {

    // Read two words from x and w
    xword = xwords[i/2];
    wword = wwords[i/2];

    // Extract values
    x1 = xword.range(31,0);
    x2 = xword.range(63,32);
    w1 = wword.range(31,0);
    w2 = wword.range(63,32);

    // Compute products
    y1 = w1*x1;
    y2 = w2*x2;

    // Write outputs
    yword.range(31,0) = y1;
    yword.range(63,32) = y2;
    ywords[i/2] = yword;
}
```

Wide bit width design

- Only one R/W to each array
- Can pipeline with II=1
- But need accesses to be contiguous